

Autonomous Tiger Racing

An introduction for prospective members

ATR



Figure 1: Nova, Auburn's AV-24, on track

Autonomous Tiger Racing is Auburn University's team for the Indy Autonomous Challenge (IAC). The goal of the challenge is to develop software that can operate an Indy NXT race car fully autonomously. Unlike in most racing series, every team runs an identical vehicle, and only the autonomy software running on it can be changed. In that sense, our work is a lot like working with a driver: reviewing their performance, giving feedback, and coaching them to improve. Because our driver is software, though, that coaching happens through vehicle dynamics analysis, identifying behaviors that can be improved, and describing those behaviors objectively enough that they can be explicitly programmed.

Most of our work follows the same cycle. We test the car and record data. We use that data to measure what the car can actually do, like how much grip the tires have or how hard it can brake. We turn those measurements into models, then use the models to plan and control the car so it uses as much of that capability as possible. Then we test again.

Our software runs on Ubuntu, a Debian-based Linux distribution, and is written primarily in C++ using ROS 2. While almost all of the code that runs on the car is ROS 2 C++, skills in standard C++, Python, and MATLAB are all useful. Linux is new to most people, so do not worry if you have not used it before. The team has development computers running Linux that members can use. On a personal computer, you can run Linux in a virtual machine (VM), through the Windows Subsystem for Linux (WSL), or by dual booting.

While our team is primarily engineers, you do not need to be an engineering major to contribute. A lot of our work is applied math and physics. Vehicle and tire dynamics are physics problems, planning the fastest lap is an optimization problem, and estimating where the car is from noisy sensors is a statistics problem. A lot of it is also computer science. Our code has to run in real time on the car, so making our solvers fast and reliable is a big part of the work. We also need people to design how our ROS 2 processes communicate, build safety checks and failsafes, and write the tools we use to command the car from the pits.

This document gives an overview of the competition, the car, and the strategies our team uses to approach

autonomous racing. Each section introduces a key part of the team or the competition without going into overwhelming detail. Terms that may be new to you are defined in the glossary in Appendix A. Contact information for specialists on each topic is provided at the end of the document.

1 Competition

The IAC hosts a few different forms of competition. Historically, the competition focused on oval tracks and consisted of two events: time trials and a passing competition. In the time trials, each team had a set amount of time to run the fastest lap it could. The passing competition was a head-to-head bracket tournament. In each matchup, the two cars took turns leading at a set speed while the other attempted a pass on the straights. If both cars completed their passes, the leader speed was raised and they went again. The first team to fail a pass lost the matchup.

In 2025, the IAC officially partnered with IndyCar, and the focus of the competition shifted to road courses. The same two events now run on road courses, with the main change being to the passing competition. Because holding a high constant speed is not practical on a road course, the cars instead run engine tunes that limit maximum power, and the power limit, rather than the leader speed, is gradually raised as the competition progresses. The only other change is that passing is now allowed anywhere on the track instead of only in designated zones.

The team has competed at venues including Indianapolis Motor Speedway, Las Vegas Motor Speedway, and Laguna Seca (Figure 2). Other venues, such as Barber Motorsports Park and Putnam Park, have been used for testing. The competition currently holds an annual race at Laguna Seca during the IndyCar race weekend, with additional testing and practice sessions throughout the year.



Figure 2: Some of the tracks where the team has raced or tested

2 AV-24 – Nova

IAC competitions use the Dallara AV-24, a modified version of the Dallara IL-15. Auburn’s AV-24 is named Nova, after one of the university’s raptors, and carries the number 34 (Figure 1). The following subsections describe the car’s sensors and control inputs.

2.1 Sensors

The AV-24 carries four types of sensors, summarized in Table 1. The GNSS/INS units tell the car where it is, and the lidars, radars, and cameras tell it what is around it.

2.2 Control Inputs

The vehicle communicates over a CAN bus, which we use both to receive vehicle data and to send commands. The inputs we can command are listed in Table 2. Every decision our software makes ends up as one of these four commands.

Table 1: Sensors on the AV-24

Sensor	Count	Make	What it provides
GNSS/INS	3	NovAtel, VectorNav	Position, velocity, and orientation
Lidar	3	Luminar	3D distance measurements all the way around the car
Radar	2	Continental, ZF	Distance and relative speed, one front and one rear
Camera	6	Allied Vision	Images, including a forward-facing stereo pair

Table 2: Commandable inputs on the AV-24

Input	Description
Steering	Steering angle and low-level PID and feedforward gains for the steering motor
Throttle	Accelerator pedal position (0–100%)
Brake	Brake pressure and front/rear brake bias
Transmission gear	Gear selection for the 6-speed sequential transmission

3 Team Algorithms

The core algorithms of most autonomous systems fall under three categories: guidance, navigation, and control (GNC). Supporting algorithms handle other needs, such as perception, stability estimation, and behavioral state machines. Figure 3 shows how these pieces fit together.

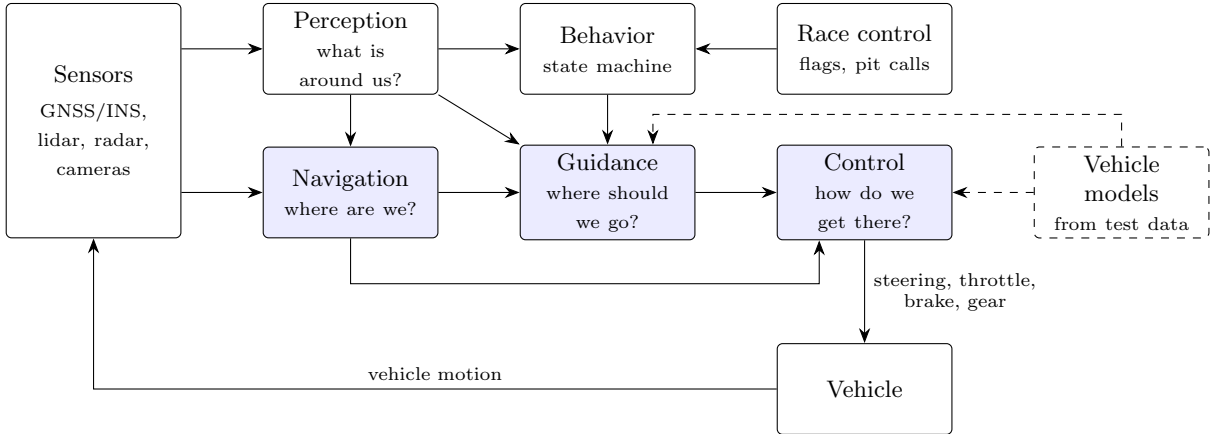


Figure 3: How the main pieces of our software connect. Shaded blocks are the core GNC algorithms. Dashed lines show vehicle models built offline from recorded test data.

Although it is not part of the code that runs on the car, one critical skill on the team is accurately interpreting data recorded during test events and deciding what should change in our models and planned behaviors. Members with an intuitive understanding of racing are essential for tuning our guidance and control algorithms so that the code captures the behavior we want.

3.1 Guidance

Within GNC, guidance provides the plan for how the system will travel from point A to point B. In its simplest form, a guidance algorithm is a path planner. Our team’s guidance work has focused on using optimization techniques to find the fastest lap the vehicle can physically drive, based on our models of the vehicle’s dynamics. The better we know the car’s limits, the closer the planned lap can get to them. As the team moves into racing alongside other vehicles, we have also started work on planning paths around other cars in real time for collision avoidance and passing maneuvers.

3.2 Navigation

Navigation tells the vehicle where it is in the world and, in some cases, what is happening around it. The classic navigation algorithm is the GPS/INS Kalman filter, which blends GPS measurements with data from inertial sensors such as IMUs to produce smooth, unbiased estimates of the vehicle’s position, velocity, and attitude (PVA). Our team’s recent navigation work has focused on handling periods when GPS measurements become poor or drop out entirely. One way we are expanding this capability is by matching camera and lidar data against maps of the track, which can keep the navigation solution accurate through these outages. Accurate navigation also matters after a run, since our measurements of what the car can do are only as good as our estimates of its state.

3.3 Control

Control closes the loop between the plan and the vehicle. It takes the motion plan from guidance, compares it against the vehicle’s current state from navigation, and determines the steering, throttle, brake, and gear commands (Section 2.2) needed to follow that plan. Historically, we have used classical control methods, but the shift to road courses has required nonlinear controllers that can handle the vehicle’s more complex behavior. Our recent nonlinear controllers have completed high-speed laps at Laguna Seca, but open problems remain. The main one is the coupling between longitudinal and lateral control. A tire can only produce a limited amount of total force, so any grip spent accelerating or braking reduces the grip available for cornering, a trade-off often visualized as the friction circle. The size of that friction circle is one of the main limits we try to measure from test data. Plotting longitudinal against lateral acceleration, called a g-g diagram, shows how much of that grip the car actually used (Figure 4). The controller needs to balance throttle, brake, and steering against each other so that the combined demand stays within what the tires can deliver.

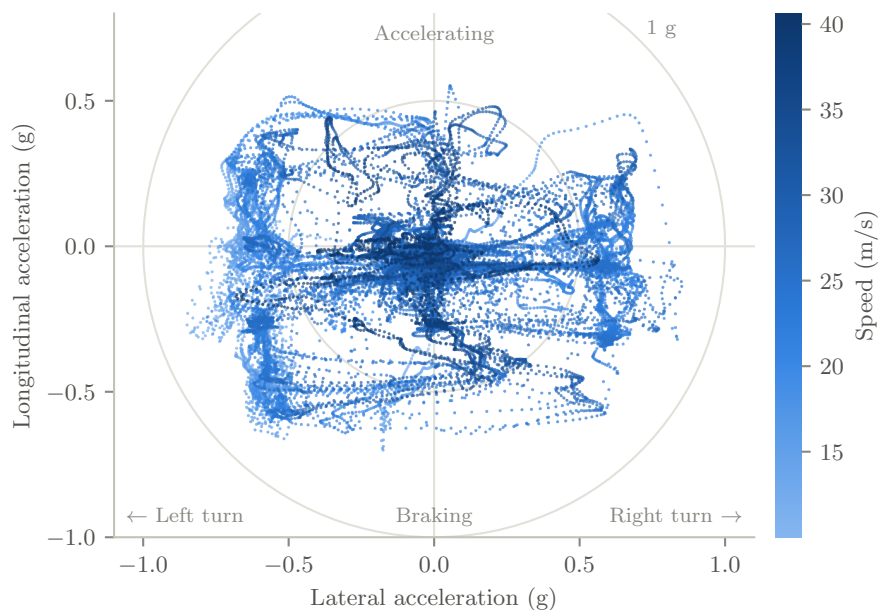


Figure 4: A g-g diagram from a 2026 test run at Laguna Seca. Each point is the car’s acceleration at one instant, colored by speed. The outer edge of the cloud shows how much grip the car used on this run.

3.4 Other Subsystems

Every piece of code on the car is critical, but not all of it fits cleanly under GNC. One major example is perception, which is closely related to navigation but is usually treated as its own field. Perception uses sensors that provide situational awareness, such as radar, lidar, and cameras, to understand what is around

the vehicle. Processing and fusing the data from these sensors is critical for reacting to other vehicles during competition, and it can also aid navigation, as noted in Section 3.2.

Other areas of work include building state machines that coordinate the behaviors required in competition (choosing when to pass, responding to commands to enter the pits, reacting to caution flags, etc.); managing communication between processes; implementing safety checks and failsafes for operational errors; and developing user interfaces that ensure every required command can be sent to the vehicle from the pits. This infrastructure is what allows any of our algorithms to actually be used in competition. These problems are more about software engineering than vehicle dynamics, so members who enjoy software architecture and writing efficient code are a big help here.

4 Workflows

Real-world testing time is very limited, so the team has developed ways to build the necessary skills early and make the most of the track time we do get.

Our primary tools between test events are three vehicle simulators. The first is NovaSim, a lightweight physics simulation that runs on almost any personal computer and is useful for early-stage development and behavioral testing. The second, made by dSPACE, is our primary simulator for team development and is also used for some competition qualifiers. It is a rough digital twin of the vehicle, with environments for Laguna Seca and Indianapolis Motor Speedway and emulations of all onboard sensors. The third, made by Automa and available on campus, models a Dallara SF23 Super Formula car. Although it simulates a different car, it gives us more practice tuning our algorithms for a new vehicle. Learning to move quickly from one simulator to another also builds the skills needed to move from simulation to the real car. While the physics in these simulators is not perfect, their models of the car’s sensors are very good. Simulation will be critical to solving the challenges of real-time vehicle detection and passing.

Auburn also has access to several other autonomous vehicles. They are not as capable as the AV-24, but they are good testbeds for gaining experience deploying algorithms on real-world systems. For instance, most of the controllers that have run on Nova were first tested on both a sedan and a Class 8 truck.

Much of our testing also happens on the bench. For example, as the team relies more on optimization algorithms built on iterative solvers, we need more testing to make sure those solvers will not fail to converge or take excessively long to solve when running on the real car. Developing maps, building perception fusion, and tuning trajectory generation parameters can all be done with nothing more than a laptop and some time.

5 Getting Started

The best way to get started is to pick a topic from this document that interests you and reach out to the specialist listed for it in Section 6. A few things will help no matter what you work on: learning C++ and ROS 2, attending simulation work sessions, and looking through data recorded at past test events.

6 Contact Information

For topics not covered by a specific individual, contact Kathleen Steadman.

Name	Email	Phone	Topics
Kathleen Steadman	kzs0138@auburn.edu	203-645-7701	Team Lead
Kyle Thompson	kat0028@auburn.edu	256-603-2486	Controls, Guidance, General Code Support
Nicholas Ott	njo0004@auburn.edu	918-704-0961	Navigation
Jamison Hood	jkh0038@auburn.edu	334-669-4122	Perception
James Brewer	jeb0176@auburn.edu	931-624-0408	Vehicle Dynamic Modeling and Analysis, Simulation

A Glossary

Term	Meaning
CAN bus	The standard communication network used in vehicles
Feedforward	A control input computed from the known plan rather than from measured error
GNC	Guidance, navigation, and control
GNSS	Global navigation satellite system. The general term for satellite positioning systems, including GPS
IMU	Inertial measurement unit. Measures acceleration and rotation rate
INS	Inertial navigation system. Uses IMU data, usually with GNSS, to estimate position, velocity, and attitude
Kalman filter	An algorithm that blends noisy measurements with a model to estimate a system's state
Lidar	A laser-based sensor that measures distance to surrounding objects
PID	Proportional-integral-derivative. A common feedback control law
PVA	Position, velocity, and attitude
Radar	A radio-based sensor that measures distance and relative speed
ROS 2	Robot Operating System 2. The software framework our code is built on